Incident Zero — Day 1 Plan

Oct 23, 2026 · @Tarun

Day 1 decides about 60 of the 100 points, not 30

Checkpoints 1 and 2 are worth 30 points directly (Eyes 10, Brain 20). But Checkpoint 4 — the 30-point hidden test on unseen incidents — is scored on the same diagnosis engine you build today, and most of the judges' 20 points (agent design, tool use, explainability, honesty) are earned by decisions you make today. Day 2 only adds the Action API layer on top.

So treat today as: build the thing that will be graded tomorrow by a machine on incidents you have never seen. Do not treat it as: get through two checkpoints.

The single highest-leverage decision today: make invented evidence structurally impossible. The brief says twice that evidence must exist in the data and that invented evidence is penalised. An LLM asked to explain an outage will confidently cite a trace ID that does not exist. Build an evidence registry (see below) — every trace ID, log line and metric value the agent cites must be one a tool actually returned in that run, checked in code, not in the prompt. This is maybe 40 lines of Python and it protects points at Checkpoints 2 and 4 and with the judges.

The second: an agent that says "I am not confident, here is what I found" scores better than a confident wrong fix. That is stated explicitly on two slides. Build the uncertain path today, not as a Day 2 afterthought — it needs a confidence value the schema can carry and a rule for when to use it.

What not to do today: do not pick a heavy agent framework. You are judged on agent design and tool use and every member must be able to explain and change the code. A plain Python tool-calling loop (about 80 lines) is faster to debug under time pressure, cheaper in tokens, and far easier to defend in the one-to-one interview than a LangGraph state machine someone pasted in at 14:00.

Before you walk in

Rule 1 says all code is written on the day and earlier commits are not allowed. So nothing below is code — it is machine state and knowledge, which is what actually eats your 12:00–13:00 hour.

On every laptop, the night before:

1. Docker with Compose v2 and Python 3.11 installed and working. Run `docker compose version` and check it prints v2.x.
2. Clone the demo and pull the images: `git clone https://github.com/open-telemetry/opentelemetry-demo.git`, then `cd opentelemetry-demo && make start`. Pulling about 14 GB of images over venue wifi at 12:05 is how teams miss Checkpoint 0.
3. Run it once end to end at home. Open the store, open `/feature`, turn on `paymentFailure` at 100%, watch the error appear in Jaeger, turn it off. Do this once with your own eyes so the demo is not new at 12:00.
4. `pip install requests` (or httpx) and your LLM provider's SDK in a Python 3.11 venv, so the installs are cached.
5. Know your RAM. Full mode needs about 6 GB; minimal mode about 3 GB but it drops Kafka, `accounting` and `fraud-detection`. See the trap about this below — it matters more than it looks.

What to read, in this order: the demo's feature-flags page (it lists every built-in fault and which service it breaks), then your provider's tool-calling / function-calling guide, then the 2-page LLM tool calling primer in the prep pack.

What to bring: at least one 16 GB laptop as the designated demo host, a phone hotspot as wifi backup, and a whiteboard marker — the architecture sketch for Checkpoint 1 can be a photo of a whiteboard, which is far faster than drawing it in a tool.

Who does what

The slides say teams of 5 but also that all three members must commit code, and that each student is assessed individually. Treat both as true: everyone commits real code, and everyone must be able to explain the whole pipeline, not just their file.

Role	Owns	Day 1 deliverable
Host	The running demo, faultctl, port discovery, keeping the stack alive	A green stack and a one-line "how to reset" command the team can run
Tools	The six telemetry tool functions, pure Python, no LLM	`tools.py` — each function callable from the CLI and returning capped, summarised JSON
Loop	The tool-calling loop, system prompt, investigation policy	`agent.py` — decides the next tool from what it has found so far
Output	JSON schema validation, the evidence registry, report rendering	`schema.py` + `report.py` — output that passes the organisers' schema every time

Role	Owns	Day 1 deliverable
Evidence	Runs the practice incidents, builds the ground-truth table, hunts hallucinations	The scoring table and the architecture sketch

Three people instead of five: merge Host into Evidence, and Output into Loop.

Two rules that keep this from falling apart. First, Tools and Loop must agree on the function signatures by 13:15 and then not change them — write the six signatures on the whiteboard at Checkpoint 0 and treat them as frozen. Second, nobody works on a branch for more than 40 minutes; merge into main often enough that a laptop dying costs one person an hour, not the team a checkpoint.

11:30–12:00 — Kickoff: capture, don't discuss

One person is the designated scribe and types everything into a shared file. Do not start designing during the reveal; you will design better at 13:00 with the facts written down. The rest of the plan assumes you walked out of kickoff with these:

- [ ] **The JSON output schema, verbatim.** Copy it into `schema.json` before you write a line of agent code. Every field name, every enum value, whether confidence is a number or a label.
- [ ] **The Action API spec.** You will not use it until Day 2, but knowing which remedies exist shapes what your diagnosis has to identify today. If the only actions are restart / toggle flag / throttle load / scale, then your root cause must map onto one of those.
- [ ] **faultctl usage.** The exact inject and reset commands, and the list of the 10 practice incident names.
- [ ] **What the "deploy history" tool actually is.** The demo has no native deploy history. The organisers add it. Find out whether it is a file, an endpoint, or a faultctl subcommand.
- [ ] **The template repository URL** and what is already in it (agent skeleton, README template, team form).
- [ ] **Where the LLM key comes from and what the spend limit is.** Ask for the number. It changes how much trace data you can afford to put in a prompt.

Ask the rules questions at the end of this document during the Q&A or at Checkpoint 0 — not in the middle of the reveal.

12:00–13:00 — Setup sprint to Checkpoint 0

Checkpoint 0 asks for five things: demo running with load, faultctl injects and resets, LLM key works, repo forked, all members committed. Run all five in parallel; none of them depends on another.

12:00–12:10, everyone, before anything else. Fork the template, everyone clones, and **everyone pushes a one-line commit now** — add your name to the team form or the README. Checkpoint 0 checks that all members have committed. Teams lose this because they plan to commit "real" code later and then one person spends the afternoon pair-programming and never pushes. Get it banked in the first ten minutes.

Then split:

Who	12:10–13:00
Host	`make start` on the designated 16 GB laptop. While images pull, open the eight URLs in the environment map below and confirm each one. Then run the port discovery commands and write the results on the whiteboard.
Tools	Hit Prometheus, Jaeger and OpenSearch with raw `curl` by hand. No Python yet. Confirm each one returns data and read the actual shape of the response.
Loop	Smoke-test the LLM key with the smallest possible tool-calling script: one tool called `get_time` that returns a string, and check the model calls it and uses the result. This proves the key, the SDK version and the tool-call parsing all work.
Output	Paste the organisers' schema into `schema.json`, `pip install jsonschema`, write a 10-line validator and a hand-written example that passes it.
Evidence	Run faultctl inject on one practice incident. Wait two minutes. Watch it appear in Jaeger and Grafana. Then reset and confirm it clears. You now know what "broken" looks like.

By 12:50 the stack must be up. If the images are still pulling at 12:40, get the USB drives the organisers brought. If RAM is the problem, switch to `make start-minimal` and tell the mentor at Checkpoint 0 — do not hide it, honesty is explicitly scored and a mentor can often free up a better laptop.

At Checkpoint 0, show the mentor a working inject-and-reset and ask your rules questions. This is the cheapest mentor time of the day; nobody else is queuing yet.

Environment map

All of this is read off the demo's own compose files, Envoy config and collector config on `main` as of today. Verify each one at 12:10 anyway — the organisers fork the repo and may pin an older version.

What	Where	Note
Web store	`http://localhost:8080/`	Envoy fronts everything on 8080
Jaeger UI	`http://localhost:8080/jaeger/ui/`	Base path is `/jaeger/ui`, not `/jaeger`
Grafana	`http://localhost:8080/grafana/`	
Load generator	`http://localhost:8080/loadgen/`	Locust UI
Flag UI	`http://localhost:8080/feature`	**Off limits for diagnosis — reading this is disqualification**
Prometheus	`http://localhost:9090`	The only backend published on a fixed host port
OpenSearch	ephemeral host port	`docker compose port opensearch 9200`
Jaeger direct	ephemeral host port	`docker compose port jaeger 16686` — or just go through 8080

The three API calls your tools are built on. Each of these works from a plain `curl` on the host:

```
# traces — list services, then search
curl "http://localhost:8080/jaeger/ui/api/services"
curl "http://localhost:8080/jaeger/ui/api/traces?
service=checkout&lookback=15m&limit=20&tags=%7B%22error%22%3A%22true%22%7D"
curl "http://localhost:8080/jaeger/ui/api/traces/<trace_id>"

# metrics
curl "http://localhost:9090/api/v1/query" --data-urlencode 'query=up'
curl "http://localhost:9090/api/v1/query_range" --data-urlencode 'query=...'
-d start=... -d end=... -d step=60

# logs (OS_PORT from the docker compose port command above)
curl "http://localhost:$OS_PORT/otel-logs-*/_search" -H 'Content-Type:
application/json' -d '{"size":20,"query":{"match_all":{}}}'
```

Metric names — verify, do not trust. The collector's `span_metrics` connector produces the RED metrics, and Prometheus's OTLP receiver renames them. They should land as `traces_span_metrics_calls_total` and `traces_span_metrics_duration_milliseconds_bucket`, with labels `service_name`, `span_name`, `span_kind`, `status_code`. Confirm in 10 seconds before building on them:

```
curl -s "http://localhost:9090/api/v1/label/__name__/values" | tr ',' '\n' |
grep -i -E 'span|container|http_check'
```

The two queries that find the broken service. Error ratio per service, and p95 latency per service:

```
sum by (service_name)
(rate(traces_span_metrics_calls_total{status_code="STATUS_CODE_ERROR"}[5m]))
  / sum by (service_name) (rate(traces_span_metrics_calls_total[5m]))

histogram_quantile(0.95,
  sum by (service_name, le)
(rate(traces_span_metrics_duration_milliseconds_bucket[5m])))
```

If `status_code` comes back as `Error` rather than `STATUS_CODE_ERROR`, fix the literal and move on — this varies by collector version, which is exactly why you run the label query first.

Container health comes from the same Prometheus. The collector scrapes Docker directly, so you get `container_*` metrics including memory usage, CPU and uptime. A container stuck in a restart loop shows as uptime repeatedly resetting to near zero — that is your main signal for the infrastructure faults the organisers inject, and it is invisible in traces.

Two timing facts that will bite you. Prometheus scrapes every 60s and the SDKs export every 60s, so a metric can lag a real fault by up to two minutes. Traces and logs are near real-time. So: inject, wait two minutes, then investigate. And Jaeger stores traces in memory capped at 25,000 — restarting the Jaeger container throws away every trace you have, so never restart it to "fix" something mid-demo.

13:00–15:30 — The agent can see

Checkpoint 1 wants a live run on one practice incident plus a one-page architecture sketch, for 10 points. The trap is spending the whole block on the LLM. Spend the first 45 minutes on plain functions with no LLM anywhere near them.

13:00–13:15 — freeze the six signatures. On the whiteboard, so Tools and Loop cannot drift:

```
list_services() -> list[str]
get_service_health(window_minutes=15) -> list[dict]   # error ratio + p95 per
service
search_traces(service, only_errors=True, min_duration_ms=None, limit=10) ->
list[dict]
get_trace(trace_id) -> dict                            # summarised span
tree, NOT raw
search_logs(service=None, contains=None, window_minutes=15, limit=20) ->
list[dict]
get_container_health(window_minutes=15) -> list[dict]  # cpu, memory, uptime,
restarts
```

Plus `get_deploy_history()` once you know what form the organisers gave it.

13:15–14:00 — build and hand-test the tools. Every function returns small, flat, already-summarised data. This is the discipline that decides whether your agent works:

- `get_trace` must **not** return raw Jaeger JSON. One trace can be 200 KB and will blow your context and your spend limit. Return a flattened list of spans: service, span name, duration in ms, status, parent, and the exception message if there is one. Thirty spans at one line each.
- Cap every list at 10–20 items and truncate every log body to about 300 characters.
- Every tool returns the time window it actually queried, so evidence can carry a timestamp.
- Each one must be runnable from the shell: `python tools.py search_traces checkout`. You will debug these fifty times today and you do not want an LLM call in that loop.

Test each against a live injected incident, not against an idle system. A tool that returns `[]` on a healthy system looks identical to a broken tool.

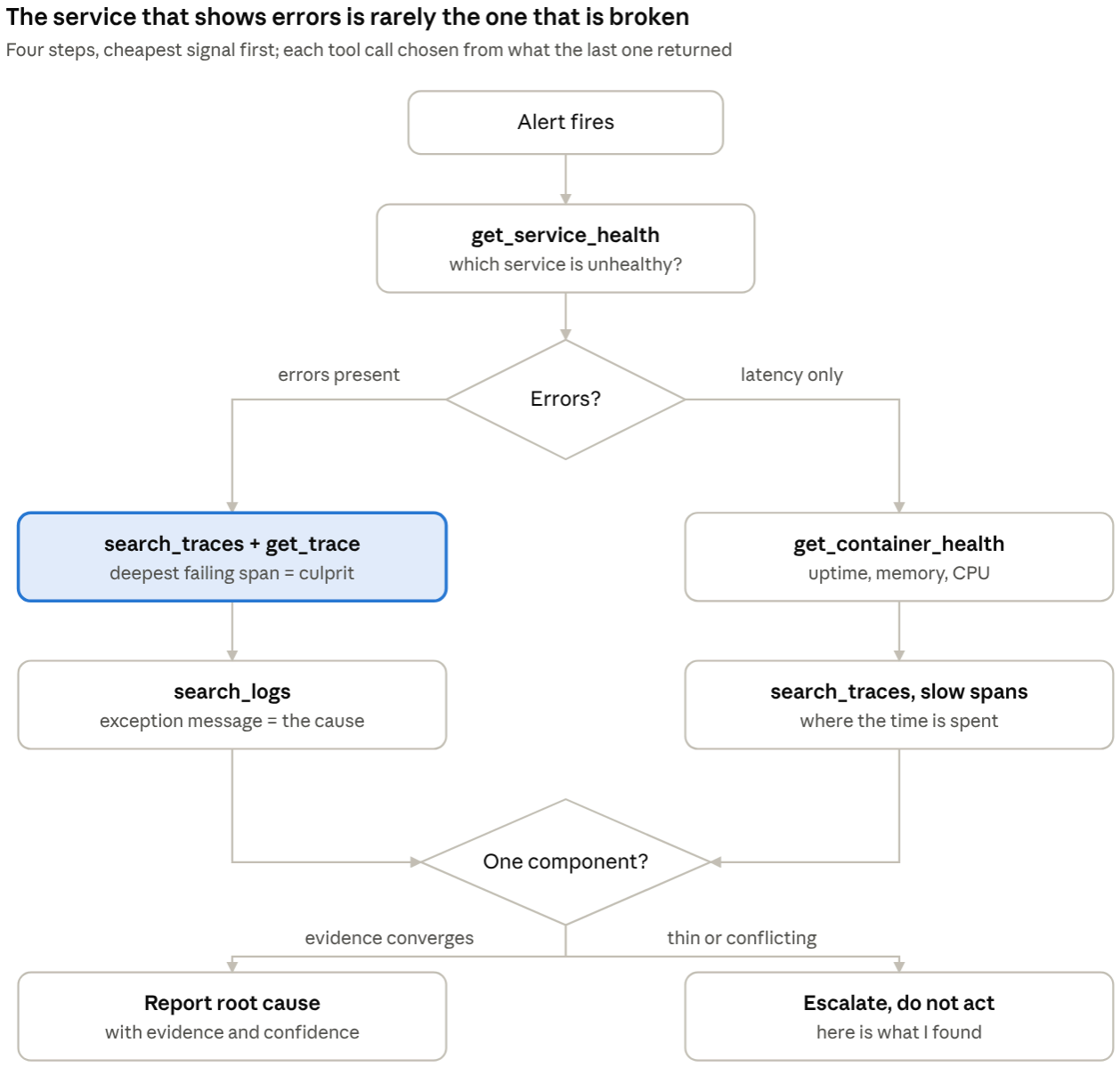
14:00–14:45 — wire the loop. Standard tool-calling loop: system prompt, user message with the alert, model replies with a tool call, you execute it, append the result, repeat until the model stops or you hit a cap of about 10 iterations. Log every tool call and result to a file — that transcript is your action log, your debugging tool, and your demo material.

At this checkpoint the agent only has to **summarise the symptoms**. Do not try to get the root cause right yet; that is the 15:30–18:30 block.

14:45–15:10 — the architecture sketch. Whiteboard, photographed. Five boxes: alert in; the agent loop in the middle; the six tools below it; Jaeger, Prometheus and OpenSearch below those; the report out. Mark the human approval gate and the kill switch as greyed-out boxes labelled "Day 2" — showing that you have already designed for them reads as planning rather than a gap.

15:10–15:30 — rehearse. Checkpoint 1 is a live run, which means it can fail live. Run it three times with a stopwatch: inject, wait two minutes, run the agent, show the output. Decide who talks and what they say. Have a terminal window already open with the command typed but not entered.

The investigation cascade



investigation cascade · 4 steps, 2 decisions

This is the policy your system prompt encodes, and it is most of what the judges mean by agent design. Write it down before you write the prompt.

Why metrics first. They are one cheap query that covers every service at once and narrows twenty services to one or two. Starting with traces means guessing which service to look at, which is exactly the guessing you are trying to remove.

Why the deepest failing span is the answer. This is the insight the whole hackathon turns on. When `paymentFailure` is on, `checkout` throws errors, `frontend` throws errors, and the metric that fires names `frontend`. None of them is broken. The broken thing is the deepest span in the trace whose status is Error and whose children are all fine — that is

`payment`. Encode this as a rule in the prompt, in plain words: *the failing component is the deepest erroring span whose own children did not error*. An agent that reports `frontend` has the symptom, not the cause, and will lose the hidden test.

Why logs last. Traces tell you *where*; the log line at that service and that timestamp tells you *what* — the exception text is usually the root cause written out in English. Always search logs filtered by the service and the time window you got from the trace, never the whole index.

The right-hand branch is how you catch the infrastructure faults. The organisers inject stopped containers, restart loops, unreachable caches and CPU starvation. Several of these produce latency or timeouts with no application exception anywhere. If there is no erroring span, the answer is almost never in the traces — go to container health and look for uptime resetting, memory pinned at the limit, or a service that has simply stopped reporting.

A useful shortcut to state in the prompt: a service that has disappeared from the metrics entirely is down, not slow. Teams miss this because an absent time series looks like no data rather than like an outage.

15:30–18:30 — The agent can diagnose

Checkpoint 2 is 20 points for a correct diagnosis with evidence and confidence on at least three practice incidents the mentor picks. The mentor picks them, so you cannot optimise for three favourites — you need breadth across the ten.

15:30–16:15 — make the output a schema-valid object, every time. Do not let the model free-write the report. Give it one final tool, something like `submit_diagnosis(root_cause_service, root_cause_summary, confidence, evidence[], proposed_action)`, and treat calling that tool as the end of the run. The arguments are already structured, so you validate against `schema.json` and either pass or ask the model to fix the one field that failed. This converts "the LLM wrote bad JSON" from a frequent failure into an impossible one.

16:15–17:30 — run the practice set and keep score. This is the most valuable 75 minutes of Day 1. Build a table and fill it in as you go:

Incident	Expected service	Agent said	Evidence real?	Tool calls	Verdict

Run all ten. Do not fix bugs as you find them — finish the sweep first, then look at the table and fix the failure that appears most often. Teams lose this block by rabbit-holing on incident 2 for an hour.

The common failures and what they mean:

- **Names the symptom service, not the cause.** Your deepest-failing-span rule is not in the prompt strongly enough, or `get_trace` is not returning the parent-child structure the model needs.
- **Right service, vague cause.** The agent stopped at traces and never pulled logs. Add a rule: do not submit until you have either a log line or a container metric, not only a trace.
- **Gets it right but takes 15 tool calls.** Your tools return too much, so the model keeps re-reading. Tighten the summaries.
- **Confident and wrong on infrastructure faults.** The right-hand branch of the cascade is missing from the prompt.

17:30–18:00 — calibration. The brief rewards an honest "not sure" over a confident wrong fix, and Day 2 has a whole stretch goal about calibrated confidence. Get the cheap version working now: a rule that the agent may only report high confidence when it has evidence from at least two of the three sources (traces, logs, metrics) pointing at the same service. One source alone caps it at medium. No source that names a specific component means escalate. Then check the table: on the incidents it got wrong, did it at least say low confidence? If it was confidently wrong, that is worse than useless and worth fixing before a new feature.

18:00–18:30 — freeze and write it down. No new features after 18:00. Clean run from scratch, everyone commits and pushes, and write the evaluation note while the results are fresh: which practice incidents it got right, which it got wrong, and why. That note is a required deliverable tomorrow and it is much harder to write at 13:00 on Day 2 from memory.

The evidence registry

Build this at about 16:00. It is roughly 40 lines and it is the thing most likely to separate you from the other teams.

The problem. Models asked to justify a conclusion produce plausible evidence. Your agent will cite trace ID `a3f9...` that it never fetched, or quote a log line that reads exactly like a real one and does not exist. The brief says invented evidence is penalised, and Checkpoint 4 is scored automatically — a fabricated trace ID is trivially detectable by the organisers' harness.

The fix, in code, not in the prompt. Every tool wraps its results before returning them: it records each trace ID, each log record ID, and each metric query plus the value it returned, into a per-run registry, and returns a short reference token alongside each item. When `submit_diagnosis` is called, you walk the evidence array and check every cited reference against the registry. Anything not there is rejected and the model is told to resubmit using only what it actually saw.

```
# one run of the agent owns one registry
evidence_registry = {}   # reference_token -> the raw item the tool returned

def register_evidence_item(kind, item):
    reference_token = kind + "-" + str(len(evidence_registry) + 1)
    evidence_registry[reference_token] = item
    return reference_token

def check_cited_evidence(cited_tokens):
    # returns the tokens the agent made up, empty list means all genuine
    invented_tokens = []
    for token in cited_tokens:
        if token not in evidence_registry:
            invented_tokens.append(token)
    return invented_tokens
```

Make each evidence item a typed object, not a sentence. A sentence cannot be checked; a record can:

Kind	Carries
trace	reference token, trace ID, service, span name, duration, status
log	reference token, timestamp, service, the actual line, truncated
metric	reference token, the exact PromQL sent, the value, the timestamp

What this buys you. The evidence in your incident report is verifiable by anyone, which is precisely what the judges' explainability and honesty criteria are asking for. It also gives you a free demo moment on Day 2: show the registry rejecting a hallucinated citation live. Very few teams will have this.

18:30–18:45 — Wrap and hand over

The milestone is "Day 1 code state saved; mentors review Day 1 progress". Fifteen minutes, so do only these:

- [ ] Everyone pushes. Verify on GitHub that all members appear in the contributor list — not just in commits you authored and they pair-programmed on.
- [ ] Tag the Day 1 state: `git tag day1-end && git push --tags`. If Day 2 goes sideways you can show something that worked.

- [ ] The scoring table goes in the repo as `evaluation-note.md`, not in someone's notebook.
- [ ] Paste the system prompt into the repo as its own file. It is the most valuable artefact you built today and the easiest to lose.
- [ ] Five lines of handover at the top of the README: what works, what does not, the one thing to fix first tomorrow.

Then plan Day 2 in five minutes, out loud, before anyone leaves. Day 2 starts at 09:00 and the whole "agent can act safely" block runs 09:30–12:00 against a 12:00 Checkpoint 3. Agree now who builds the Action API wrapper, who builds the approval gate and kill switch, and who builds the recovery verification, so nobody arrives tomorrow wondering what they own.

One thing worth deciding tonight: your approval gate can be a terminal prompt. It does not need a UI. The chat-style approval UI is listed as an optional stretch goal, not a requirement, and a `input("approve? [y/N] ")` that is logged satisfies "nothing changes without an explicit yes" completely.

Traps, and what to ask the mentor

The disqualification trap, first. Rule 6 says reading feature-flag settings to diagnose means disqualification. The obvious version is easy to avoid: no tool hits `/feature`, flagd, or `demo.flagd.json`. The non-obvious version is that OpenTelemetry emits feature-flag *telemetry* — spans and log records can carry `feature_flag.key` attributes naming the flag that was evaluated. That is telemetry, not settings, but it also hands your agent the answer. **Ask the mentor at Checkpoint 0 whether `feature_flag.*` attributes in traces and logs are allowed as evidence.** Until you have an answer, strip those attributes in your tool layer. It costs you nothing — the exception messages and container metrics are enough — and getting this wrong late costs you everything.

The minimal-mode trap. `make start-minimal` drops Kafka, `accounting` and `fraud-detection`. Two of the demo's built-in faults (`kafkaConsumerDead`, `kafkaQueueProblems`) live entirely in those services. If the mentor picks one of those at Checkpoint 2 and your stack does not run them, you cannot diagnose it at all. Run full mode on at least one 16 GB laptop and make that machine the one you demo from, even if everyone else develops against minimal.

The other time sinks:

Trap	Cost	Avoid it by
Metrics lag the fault by up to 2 minutes	30 min of "the tool is broken"	Inject, wait 2 minutes, then query
Jaeger is at `/jaeger/ui/`, not port 16686	20 min	Use the URLs in the environment map
OpenSearch has no fixed host port	20 min	`docker compose port opensearch 9200`
`get_trace` returning raw JSON	Blown context and spend limit	Flatten to one line per span before returning
Restarting Jaeger to "fix" something	Every trace you had	Never restart Jaeger mid-session
One person pair-programming all day	A Checkpoint 0 item	Everyone commits at 12:10

Questions to ask at Checkpoint 0 (write the answers down):

1. Are `feature_flag.*` attributes in telemetry allowed as evidence, or treated as reading flag settings?
2. The milestone slide says code freeze at 14:00 on Day 2; the deliverables slide says 16:30. Which is it?
3. Teams of 5, but the rules say all three members must commit — is it all members, whatever the team size?
4. Is the hidden test scored on the JSON schema fields only, or is the human-readable report scored too?
5. For the hidden test, does the harness inject the incident and then call our agent, or does our agent have to detect it on its own?
6. What exactly is the deploy-history tool, and does the hidden test use it?

Question 5 matters more than it looks: if the harness calls you, your entry point needs to be a clean function or CLI command that takes an alert and returns the JSON. Build that shape from the start today and tomorrow's integration is free.